

Neo4j TestPilot: Project Walkthrough (STAR Method)

This document provides a detailed walkthrough of the Neo4j TestPilot project, following the STAR (Situation, Task, Action, Result) framework for a clear and comprehensive overview suitable for technical interviews.

Project Overview

Project Name: Neo4j TestPilot

One-Line Description: An AI-assisted code generation system that transforms legacy semiconductor test codebases into a Neo4j knowledge graph, enabling GitHub Copilot to automatically generate production-ready test methods.

Situation

Pain Point: The existing semiconductor test program codebase, comprising over 481 Java classes, was complex and lacked proper documentation. This made it difficult for engineers to understand class relationships, dependencies, and the overall architecture. Manual test method creation was a time-consuming and error-prone process, taking several hours for each test.

Requirement: There was a critical need to automate the test generation process, improve code discoverability, and ensure consistency across different versions of the codebase. The system needed to be intelligent enough to understand the semantic meaning of the code, not just its syntax.

Why it was needed: To significantly reduce the time and effort required for test development, minimize human error, and create a scalable and maintainable testing framework. This would free up engineering resources to focus on more complex and value-added tasks.

Task

My Role: As the lead developer, I was tasked with designing and implementing the entire Neo4j TestPilot system from the ground up.

What I was tasked to solve: The primary objective was to build an intelligent system that could ingest the entire Java test program codebase, create a comprehensive knowledge graph, and integrate it with an AI code generation tool (GitHub Copilot) to automate test method creation. The system had to be robust, scalable, and easy to use.

Action

I architected and implemented a multi-stage solution that involved three key pillars: a robust ingestion system, a powerful knowledge graph, and a seamless AI integration.

1. 8-Phase Knowledge Graph Ingestion System

I designed an 8-phase ingestion pipeline to parse the Java codebase and populate the Neo4j graph database. This modular approach ensured maintainability and allowed for targeted optimizations.

Phase	Description
1. Project Structure	Parsed <code>build.xml</code> to identify project modules and external JAR dependencies.
2. Class Parsing	Utilized the Tree-sitter parsing framework to achieve a 100% success rate in parsing all 481 Java classes, including those with complex or even broken syntax.
3. Members Extraction	Extracted detailed information from each class, including methods, fields, annotations, and signatures.
4. Dependency Graph	Mapped the intricate web of dependencies between classes, packages, and external JARs.
5. Import Resolution	Resolved all import statements to create explicit links between classes and their dependencies.
6. Git Versioning	Integrated with Git to track code changes across different commits, tags, and branches, enabling version-specific analysis.
7. Semantic Changes	Implemented an LLM-powered semantic change detection system to identify breaking changes between code versions.
8. Markdown Linking	Automatically linked Markdown documentation files to the corresponding code entities in the graph.

2. MCP Integration for AI-Assisted Code Generation

To bridge the gap between the Neo4j knowledge graph and GitHub Copilot, I implemented a **Model Context Protocol (MCP)** server.

- **GenAI Toolbox:** I built a Go-based MCP server that exposed the Neo4j database as a set of queryable tools.
- **Custom Tools:** I defined custom MCP tools in YAML, such as `get_datas_from_particular_ingestion_version` and `search_class_by_name`, allowing Copilot to interact with the graph using natural language queries.
- **VS Code Integration:** I configured VS Code to connect to the MCP server, enabling a seamless workflow where engineers could generate code directly within their IDE.

3. Semantic Change Detection

To address the challenge of tracking breaking changes between versions, I developed a semantic analysis module that used an LLM (OpenAI) to analyze `git diff` outputs. This system could automatically identify:

- Method signature changes
- Import statement removals
- Variable type changes

This proactive approach to change detection prevented potential runtime errors and significantly reduced the manual effort required for code reviews.

Result

Quantitative Impact

Metric	Achievement
Test Generation Time	Reduced from 2-3 hours to < 10 minutes (an 18x improvement).
Code Quality	Achieved zero linting errors in generated code, compared to 5-10 per manually written file.
Parse Success Rate	100% (⁴⁸¹ / ₄₈₁ classes) with the Tree-sitter parser.
Documentation	100% Javadoc coverage in generated code.

Business Value

- **Increased Productivity:** The 18x reduction in test generation time freed up significant engineering resources.
- **Improved Code Quality:** The automated generation of high-quality, fully documented code reduced technical debt and improved maintainability.
- **Enhanced Knowledge Management:** The Neo4j knowledge graph became a central source of truth” for the codebase, enabling engineers to quickly

understand complex relationships.

- **Risk Mitigation:** The automated breaking change detection system proactively identified potential issues, reducing the risk of production bugs.

Technical Stack Summary

Category	Technologies
Languages	Python, Java, Go, Cypher, YAML, JSON
Database	Neo4j
AI/ML	OpenAI API, GitHub Copilot, Model Context Protocol (MCP)
Frameworks/Tools	Tree-sitter, Git, VS Code, GenAI Toolbox